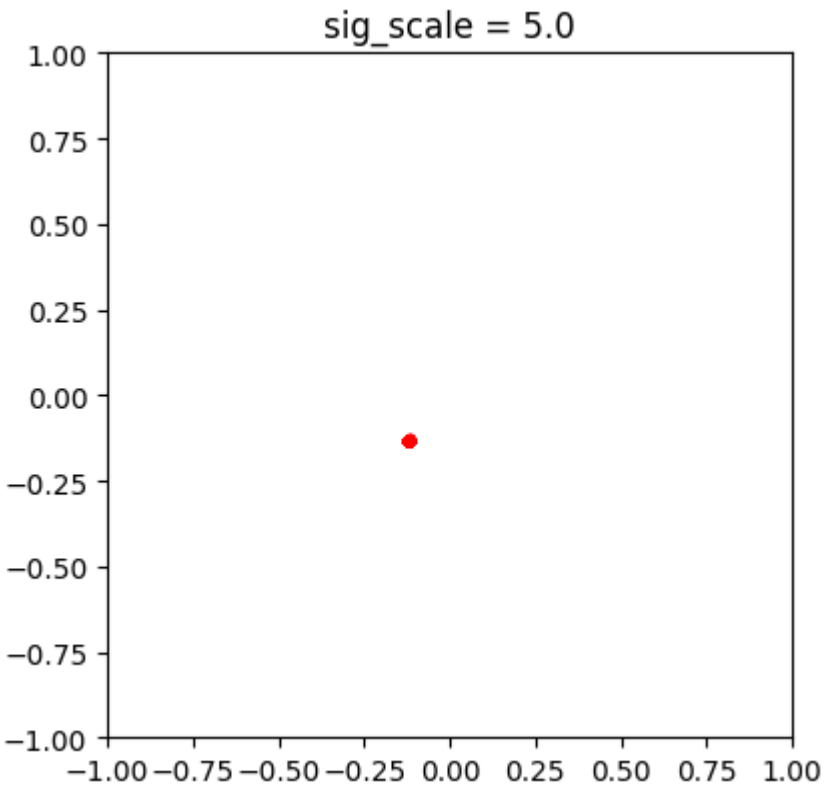


A/B Assignment

With your best model please investigate at which number of update steps in the gnerate process the performance begins to really break down, that is F_1 of the generated samples falls below .9 (ie Precision < .8). Your challenge is to train a de-noising model that can get a F1 score > with <= 6 function evalations (In the code for each update step you actually evaluate the function twice due to having the score_fn and vf_fn defined seperately. They both rely on the noise_pred_fn and thus one call to noise_pred_fn is all that is really needed. But for pedagogical reasons I have defined them separately. But for this assignment count an evaluation of both the score_fn and vf_fn as one NFE.) You can solve this challenge in any way you like. But here are some hints of approaches that could be combined or explored:

1. One interesting and straightforward approach is to train the model with idea in this paper **ONE STEP DIFFUSION VIA SHORTCUT MODELS** (https://arxiv.org/pdf/2410.12557). Here an extra self-consistency loss is added during training to promote learning a

vector-field on the desired step-size of the solver and thus allows the trained model to skip ahead at generation time. It would be fun to use some of the ideas in this paper to help with training a network effective at higher step sizes.

2. One successful upgrade to the basic flow matching is re-flow. A simple iterative procedure introduced in **Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow** (https://arxiv.org/pdf/2209.03003) to promote learning straight paths that transports the initial distribution to the data distribution. Initially, standard flow matching, with training paths computed from linear interpolations of independently matched end-points, is used to train a neural network to transport p_{init} to p_{data}. This neural network is then used to generate from $x_0 \sim p_{\text{init}}$ a point $x_1 \sim p_{\text{data}}$ and these points are put in correspondence. This is repeated n times and these corresponding points are then used to generate different x_t via linear interpolation and sampling of t. A new neural network is then trained via flow matching using this new matched data. These explicitly matched noise and data points used in the new training should promote straighter resulting flows. You can iteratively repeat the re-training process multiple times by creating new corresponding points with the newly trained neural network from the previous iteration.
3. Customize the noise scheduler you use for training
4. Upgrade the sde solver to something such as stochastic Heun or a simple ODE solver with adaptive step size. Though the latter may only give efficiency gains if the flow trajectories defined by vf_fn are

In my work below I implement progressive distillation inspired by: **PROGRESSIVE DISTILLATION FOR FAST SAMPLING OF DIFFUSION MODELS** https://arxiv.org/pdf/2202.00512
I am able to get an F_score ≈ 0.94 with just 6 NFE.
I am also able to get good perceptual quality (see toward the end) with <=6 NFE.
The results are compared to three baseline models which gets F_score $\approx 0.74 - 0.86$ with 6 NFE and the best model encountered with F_score ≈ 0.97 using 48 NFE.
First, I establish a baseline and investigate the problem, a simple form of progressive distillation is then implemented, refined, and tested.
Enjoy!

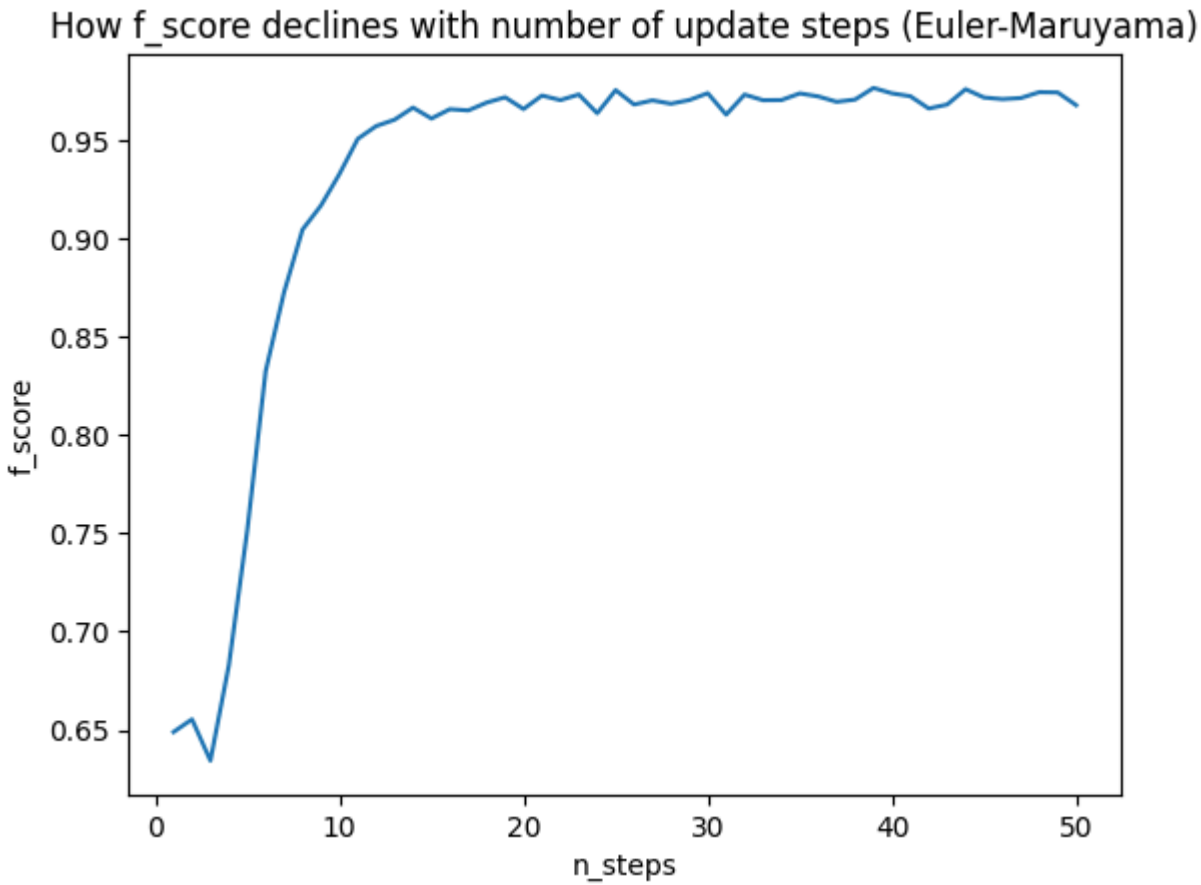
Let us start by investigating when the performance of the generation process breaks down (F_1 below 0.9). We start by looking at the model trained with a linear scheduler where sampling is done using the Euler-Maruyama update equation.

In [12]:
```
with open('noise_pred_fn.pkl', "rb") as f:
    test_noise_pred_fn = pickle.load(f)
    test_score_fn = pickle.load(f)
    test_vf_fn = pickle.load(f)
```

```
    test_scheduler = pickle.load(f)

p_data = a3_utils.CheckerboardSampleable(device, grid_size=4)
```

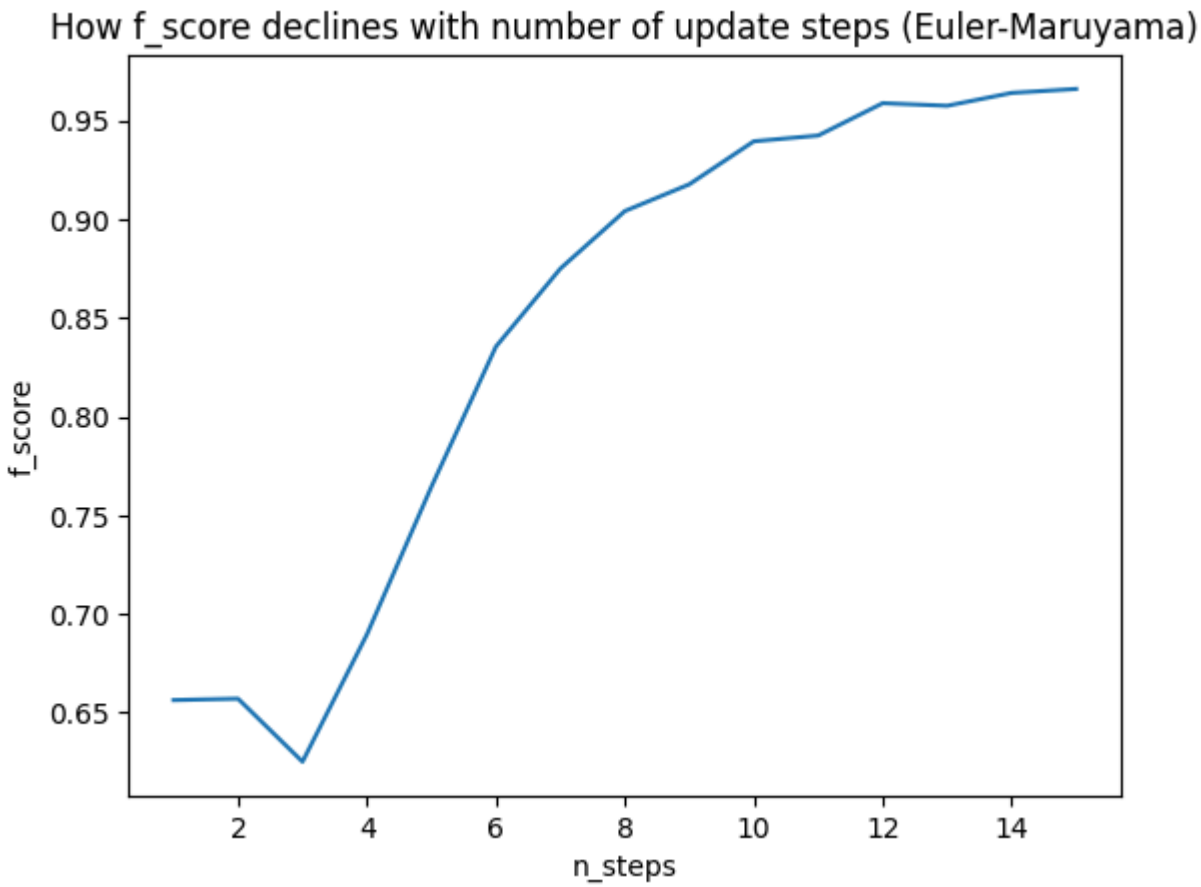
In [13]:

```
n_try = 2000
step_list = np.arange(1, 51)
f_score_list = np.zeros_like(step_list, dtype=float)

for i, steps in enumerate(step_list):
    temp_gen = GenerateSamples(update_step_euler_maruyama, test_vf_fn, sde_flag=True, score_fn=test_score_fn, sigma_fn=test_sc
    _, _, temp_f_score = measure_performance2(temp_gen, p_data, n_try, n_steps=steps)
    f_score_list[i] = temp_f_score

plt.plot(step_list, f_score_list)
plt.title('How f_score declines with number of update steps (Euler-Maruyama)')
plt.xlabel('n_steps')
plt.ylabel('f_score')
plt.show()
```



In [14]:
```
n_try = 2000
step_list = np.arange(1, 16)
f_score_list = np.zeros_like(step_list, dtype=float)

for i, steps in enumerate(step_list):
    temp_gen = GenerateSamples(update_step_euler_maruyama, test_vf_fn, sde_flag=True, score_fn=test_score_fn, sigma_fn=test_sc
    _, _, temp_f_score = measure_performance2(temp_gen, p_data, n_try, n_steps=steps)
    f_score_list[i] = temp_f_score

plt.plot(step_list.astype(np.int64), f_score_list)
plt.title('How f_score declines with number of update steps (Euler-Maruyama)')
plt.xlabel('n_steps')
```

```
plt.ylabel('f_score')
plt.show()
```



We see that the model starts breaking down at around 14 update steps and falls below the 0.9 f_score threshold at around 8 update steps.

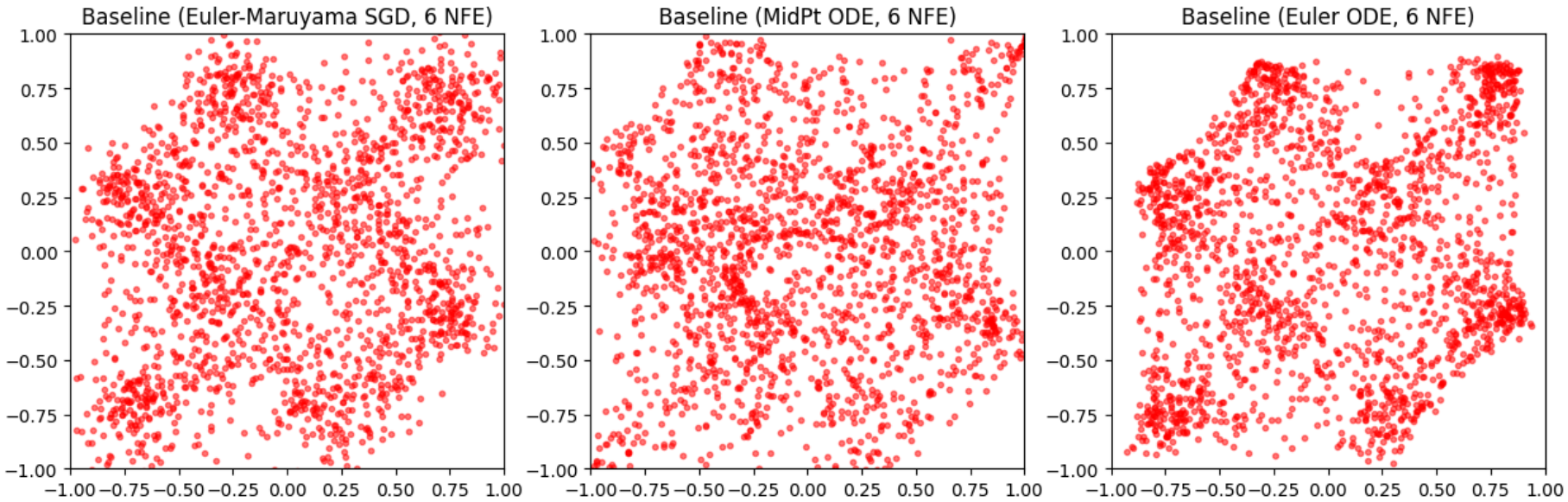
Let us establish a baseline by using one of our models from part 1 and part 2. Namely, the model that was trained using a linear scheduler and 30000 update steps. We set the number of update steps during generation process to n_steps = 6 and evaluate the generated points and also performance metrics.

That precision is not very impressive but will be a good baseline to beat. Remember the midpoint solver from one of the earlier exercises. Let us also see what performance we get using midpoint ODE sampling with 3 update steps (corresponding to 6 NFE).

In [15]:

```
# Baseline: Euler-Maruyama SGD
n_steps = 6
baseline_gen = GenerateSamples(update_step_euler_maruyama, test_vf_fn, sde_flag=True, score_fn=test_score_fn, sigma_fn=test_sc
X_t = baseline_gen.generate(X_0, n_steps=n_steps)

print(f'\n Baseline (Euler-Maruyama SGD, {n_steps} NFE)')
measure_performance(baseline_gen, p_data, n_try, n_steps = n_steps)

fig = plt.figure(figsize=(15, 5))
axes = fig.add_subplot(1, 3, 1)
scatter_plot_of_points(X_t, f'Baseline (Euler-Maruyama SGD, {n_steps} NFE)', axes)

# MidPT ODE
gen_samples_ode_midpt = GenerateSamples(update_step_midpoint_ode, test_vf_fn, sde_flag=False)
n_steps = 3
X_t_midpt = gen_samples_ode_midpt.generate(X_0, n_steps=n_steps)

print(f'\n Baseline (MidPt ODE, {n_steps*2} NFE)')
measure_performance(gen_samples_ode_midpt, p_data, n_try, n_steps = n_steps)

axes = fig.add_subplot(1, 3, 2)
scatter_plot_of_points(X_t_midpt, f'Baseline (MidPt ODE, {n_steps*2} NFE)', axes)

# Euler ODE
gen_samples_ode_euler = GenerateSamples(update_step_euler_ode, test_vf_fn, sde_flag=False)
n_steps = 6
X_t_euler = gen_samples_ode_euler.generate(X_0, n_steps=n_steps)

print(f'\n Baseline (Euler ODE, {n_steps} NFE)')
measure_performance(gen_samples_ode_euler, p_data, n_try, n_steps = n_steps)

axes = fig.add_subplot(1, 3, 3)
scatter_plot_of_points(X_t_euler, f'Baseline (Euler ODE, {n_steps} NFE)', axes)
```

```
 Baseline (Euler-Maruyama SGD, 6 NFE)
Precision: 0.7085
Normalized Entropy: 0.9982
Harmonic Mean (F-score): 0.8288

 Baseline (MidPt ODE, 6 NFE)
Precision: 0.6110
Normalized Entropy: 0.9809
Harmonic Mean (F-score): 0.7530

 Baseline (Euler ODE, 6 NFE)
Precision: 0.7555
Normalized Entropy: 0.9975
Harmonic Mean (F-score): 0.8598
```



Not very impressive but interestingly, adding just two more NFE for the midpoint sampler we get a quite good F-score (> 0.92) so there is some potential in the Mididpoint ODE sampling.

In [26]:
```
n_steps=4
print(f'\n Performance of the Deterministic sampling with MidPt ODE and n_steps={n_steps} ({n_steps*2} NFE)')
temp = measure_performance(gen_samples_ode_midpt, p_data, n_try, n_steps = n_steps)
```

```
 Performance of the Deterministic sampling with MidPt ODE and n_steps=4 (8 NFE)
Precision: 0.8620
Normalized Entropy: 0.9949
Harmonic Mean (F-score): 0.9237
```

Let us now look at the model which use our damped cosine scheduler from the previous exercise and see if it has a similar breaking point.

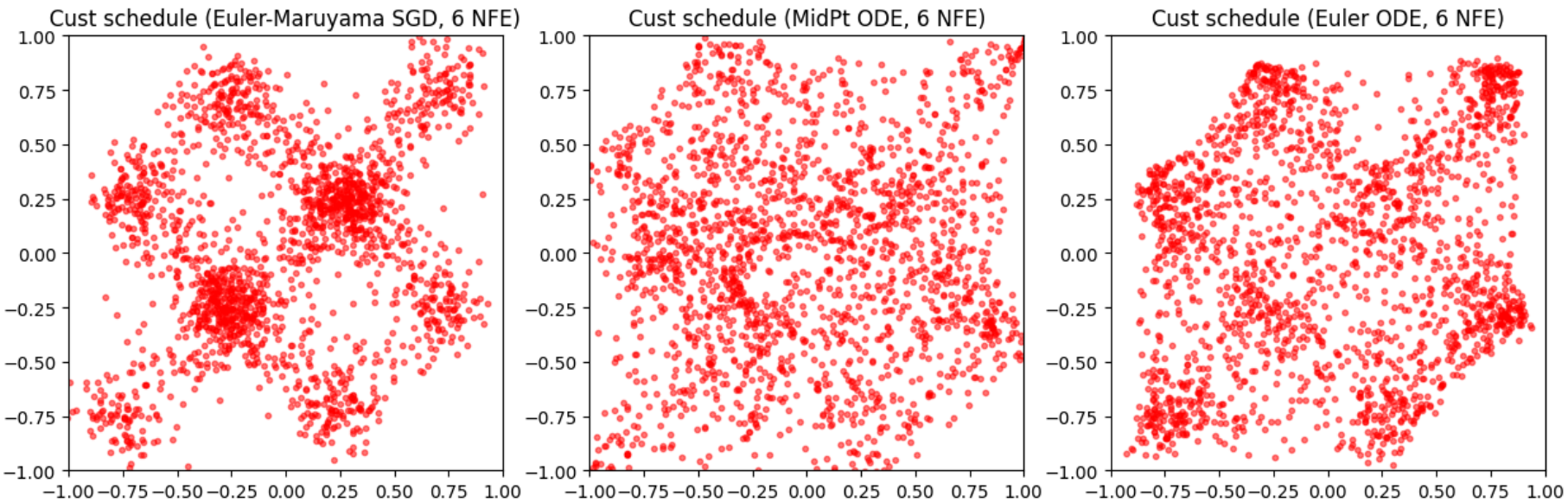
In [16]:
```
if __name__ == '__main__':
    # up the number of training iterations to train a decent model
    ## set-up the scheduler
    cos_alpha_fn = AlphaFn(sin_fn, sin_fn_dt)
    damped_beta_fn = BetaFn(damped_cos_fn, damped_cos_fn_dt)
    custom_scheduler = Scheduler(cos_alpha_fn, damped_beta_fn)
    n_update = 30000
    uniform_sampling = TimeSampling('uniform')
    network_params = {'L': 4, 'h': 64}
    print(f'Training network to predict noise with a custom scheduler and {n_update} update steps.....')
    training_params = {'n_batch': 1024, 'n_update': n_update, 'time_sampling': uniform_sampling}
    custom_noise_pred_fn, custom_score_fn, custom_vf_fn = set_up_and_perform_training(network_params, training_params, custom_
                                                                                       noise_target_fn, 'noise_pred', p_data)
```

```
Training network to predict noise with a custom scheduler and 30000 update steps.....
Training 30000 steps in 197.8391 seconds
```

In [18]:
```
# Euler-Maruyama SGD
n_steps = 6
custom_sgd_gen = GenerateSamples(update_step_euler_maruyama, custom_vf_fn, sde_flag=True, score_fn=custom_score_fn, sigma_fn=c
X_t = custom_sgd_gen.generate(X_0, n_steps=n_steps)

print(f'\n Cust schedule (Euler-Maruyama SGD, {n_steps} NFE)')
measure_performance(custom_sgd_gen, p_data, n_try, n_steps = n_steps)

fig = plt.figure(figsize=(15, 5))
axes = fig.add_subplot(1, 3, 1)
scatter_plot_of_points(X_t, f'Cust schedule (Euler-Maruyama SGD, {n_steps} NFE)', axes)

# Midpoint ODE
custom_gen_ode_midpt = GenerateSamples(update_step_midpoint_ode, custom_vf_fn, sde_flag=False)
n_steps = 3
X_t_midpt = gen_samples_ode_midpt.generate(X_0, n_steps=n_steps)
```

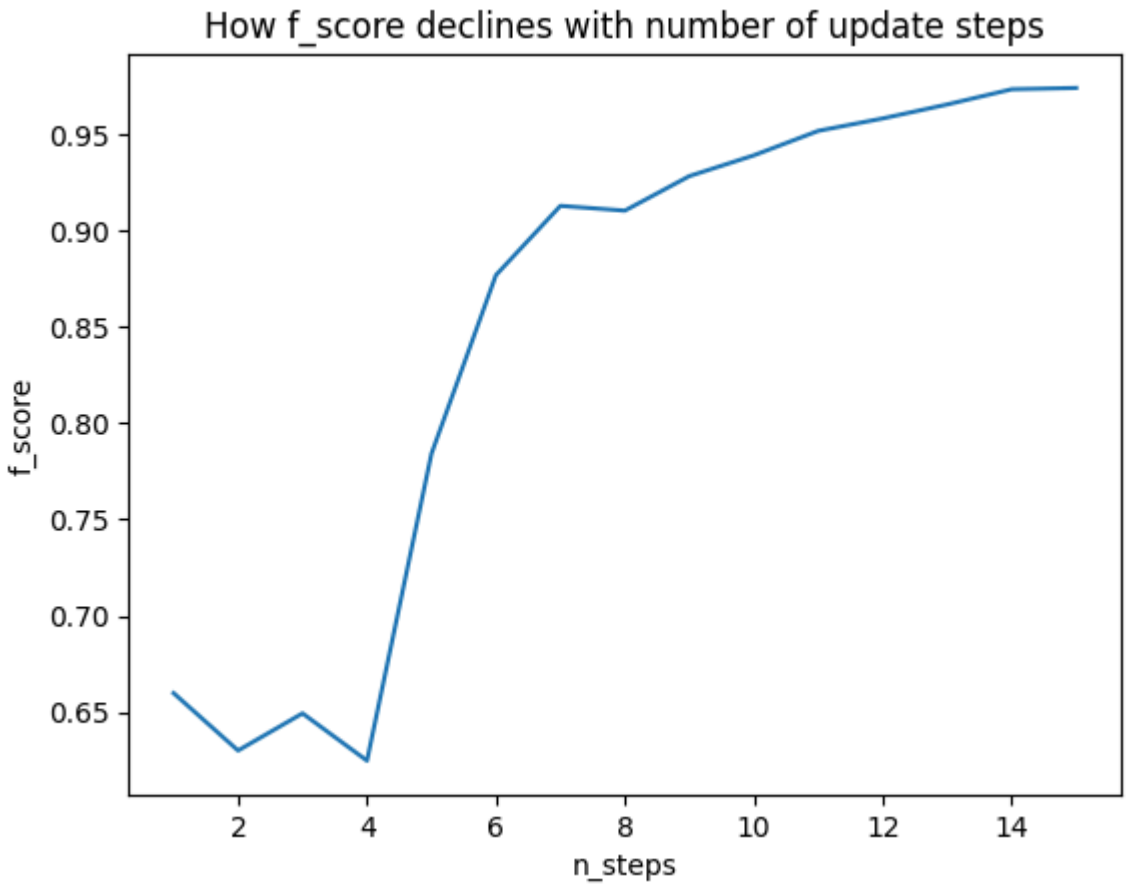
```python
print(f'\n Cust schedule (MidPt ODE, {n_steps*2} NFE)')
measure_performance(custom_gen_ode_midpt, p_data, n_try, n_steps = n_steps)

axes = fig.add_subplot(1, 3, 2)
scatter_plot_of_points(X_t_midpt, f'Cust schedule (MidPt ODE, {n_steps*2} NFE)', axes)

# Euler ODE
custom_gen_ode_euler = GenerateSamples(update_step_euler_ode, custom_vf_fn, sde_flag=False)
n_steps = 6
X_t_euler = gen_samples_ode_euler.generate(X_0, n_steps=n_steps)

print(f'\n Cust schedule (Euler ODE, {n_steps} NFE)')
measure_performance(custom_gen_ode_euler, p_data, n_try, n_steps = n_steps)

axes = fig.add_subplot(1, 3, 3)
scatter_plot_of_points(X_t_euler, f'Cust schedule (Euler ODE, {n_steps} NFE)', axes)
```

```
 Cust schedule (Euler-Maruyama SGD, 6 NFE)
Precision: 0.8490
Normalized Entropy: 0.9245
Harmonic Mean (F-score): 0.8851

 Cust schedule (MidPt ODE, 6 NFE)
Precision: 0.7045
Normalized Entropy: 0.5647
Harmonic Mean (F-score): 0.6269

 Cust schedule (Euler ODE, 6 NFE)
Precision: 0.9185
Normalized Entropy: 0.8579
Harmonic Mean (F-score): 0.8872
```



In [ ]:

```
n_try = 2000
step_list = np.arange(1, 16)
f_score_list = np.zeros_like(step_list, dtype=float)

for i, steps in enumerate(step_list):
    temp_gen = GenerateSamples(update_step_euler_maruyama, custom_vf_fn, sde_flag=True, score_fn=custom_score_fn, sigma_fn=cus
    _, _, temp_f_score = measure_performance2(temp_gen, p_data, n_try, n_steps=steps)
    f_score_list[i] = temp_f_score

plt.plot(step_list.astype(np.int64), f_score_list)
plt.title('How f_score declines with number of update steps (Euler-Maruyama)')
plt.xlabel('n_steps')
plt.ylabel('f_score')
plt.show()
```



Training with a damped cosine scheduler gives slightly better performance for lower NFE during generation compared to a model training with a linear scheduler. Interesting, let us keep this in mind. I will still use the linear scheduler for my improved model below to avoid potential undesired behaviour because of the scheduler. It is also interesting to see what performance a simple model can achieve with principles adopted for more advanced and complicated samplers etc.

Let us switch focus and see if we can create a model that achieves better performance than above (but still 6<=NFE) using a simple model and applying some of the techniques below:

Progressive Distillation for Fast Sampling of Diffusion Models

In the original paper they distill the behaviour of a N-step DDIM sampler for a pretrained diffusion model into a new model with N/2 steps. Repeating this process recursively mean we can distill the behaviour of the N-step sampler into a more compact (N/2^k)-sampler for some integer k dependent on the number of recursive calls.

I implement a simple version of the above, appropriate for our toy problem. Starting by simply distilling our N-step Euler ODE sampler using our pretrained diffusion model trained with a linear scheduler and 30000 update steps.

First, let us set up our help function for training, similarly to the given function in this assignment. Note that I initialize the student as a copy of our pretrained model to speed up convergence. This means that I am using the same network architecture for the student as we used to train the teacher (MLP model from part 1 and 2) which is what they do in the paper.

In [19]:
```
def set_up_and_perform_student_training(network_params, training_params, scheduler, teacher_vf, teacher_noise, p_data):
    start = time.time()

    student = copy.deepcopy(teacher_noise) # MLP_fn(network_params) # Can also uncomment this line to use a different MLP arch

    # New trainer class (but this one is pretty similar to the one used in the assignment)
    trainer = distill_mlp_fn(student, scheduler, teacher_vf, training_params)
    student = trainer.train_network(p_data)

    end = time.time()
    print(f"Distillation training {training_params['n_update']} steps in {end - start:.4f} seconds")

    # Will return score and vector field as well
    score_fn = ScoreFn(student, 'noise_pred', scheduler=scheduler)

    X = p_data.sample(1000)
    mean_x1 = torch.mean(X, dim=0)
    vf_fn = VectorFieldFn(student, 'noise_pred', scheduler=scheduler, data_mean=mean_x1)

    return student, score_fn, vf_fn
```

In [20]:
```python
class distill_mlp_fn:

    def __init__(self, mlp, scheduler, teacher_vf_fn, training_params):
        self.mlp = mlp
        self.scheduler = scheduler
        self.teacher_vf_fn = teacher_vf_fn

        self.n_batch = training_params['n_batch']
        self.n_update = training_params['n_update']
        self.time_sampling = training_params['time_sampling']
        self.teacher_N = training_params['teacher_N']

    def train_network(self, p_data, device='cpu'):
        optimizer = torch.optim.Adam(self.mlp.parameters(), lr=1e-3)
        loss_fn = torch.nn.MSELoss()

        for step in range(self.n_update):
            x_1 = Tensor(p_data.sample(self.n_batch).to(device))
            x_0 = torch.randn_like(x_1[:,0:2])

            idx = torch.randint(1, self.teacher_N+1, (len(x_1),), device=device)
            t = idx / self.teacher_N
            t = t.unsqueeze(1).to(device)

            a_t, b_t = self.scheduler(t)
            x_t = a_t * x_1 + b_t * x_0

            optimizer.zero_grad()

            # Teacher prediction
            h = 1/self.teacher_N
            with torch.no_grad():
                x_mid = update_step_euler_ode(x_t, t, h, self.teacher_vf_fn)
                teacher_pred = update_step_euler_ode(x_mid, t+h, h, self.teacher_vf_fn)

            # Student prediction
            mean_x1 = torch.mean(x_1, dim=0)
            student_vf_fn = VectorFieldFn(self.mlp,'noise_pred', self.scheduler, mean_x1)
            student_pred = update_step_euler_ode(x_t,t,2*h,student_vf_fn)
```

```
            # Loss: match teacher
            loss = loss_fn(student_pred, teacher_pred)
            loss.backward()
            optimizer.step()

            if step % 999 == 0:
                print(f'[Distill step {step+1}] loss = {loss.item():.7f}')

        return self.mlp
```

We now have code that works for 1 distilling "iteration". Which mean that we can distill our Euler ODE sampler from N steps to N/2 steps. Let us start by trying if we already can beat the threshold performance by training our student to sample using 6 steps (6 NFE) by training with a teacher that samples using 12 steps (12 NFE). Later we will of course also apply this recursively, so stay tuned below!

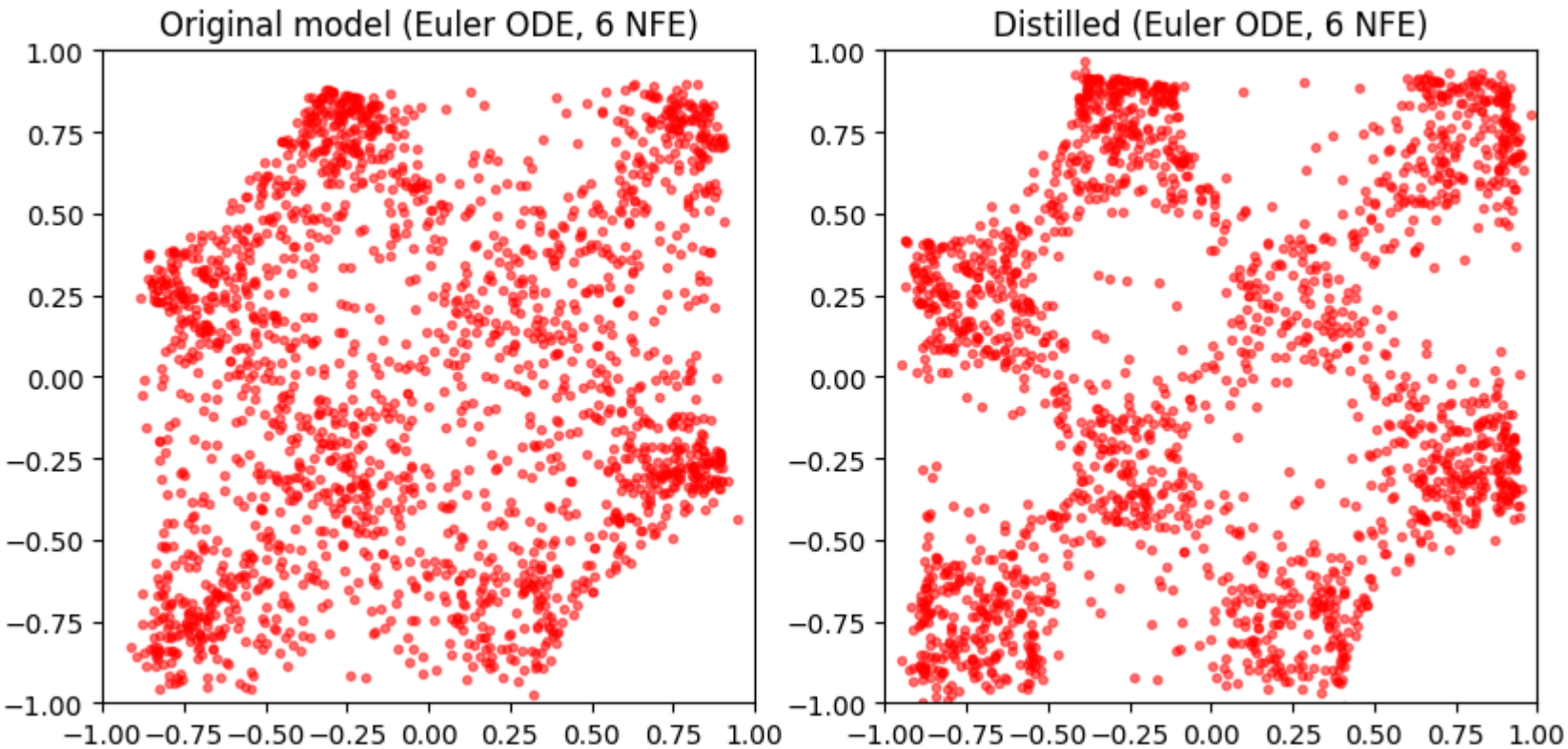
In [64]:
```
uniform_sampling = TimeSampling('uniform')
network_params = {'L': 4, 'h': 64}
training_params = {'n_batch': 1024, 'n_update': 2000, 'time_sampling': uniform_sampling, 'teacher_N': 12}

linear_scheduler = Scheduler.from_named_scheduler('Linear')
student_noise, student_score, student_vf = set_up_and_perform_student_training(
    network_params, training_params, linear_scheduler,
    teacher_fn=test_vf_fn, teacher_noise=test_noise_pred_fn, p_data=p_data
)
```

```
[Distill step 1] loss = 0.0004493
[Distill step 1000] loss = 0.0000132
[Distill step 1999] loss = 0.0000306
Distillation training 2000 steps in 23.3581 seconds
```

In [ ]:
```
# Cell that plots distilled and original sampling techniques next to each other
teacher_gen = GenerateSamples(update_step_euler_ode, test_vf_fn, sde_flag=False)
n_try = 2000
n_steps = 6
X_teacher = teacher_gen.generate(X_0, n_steps=n_steps)

print(f'\n Original model (Euler ODE, {n_steps} NFE)')
measure_performance(teacher_gen, p_data, n_try, n_steps = n_steps)

student_gen = GenerateSamples(update_step_euler_ode, student_vf, sde_flag=False)
X_student = student_gen.generate(X_0, n_steps=n_steps)
```

```python
print(f'\n Distilled (Euler ODE, {n_steps} NFE)')
measure_performance(student_gen, p_data, n_try, n_steps = n_steps)

fig = plt.figure(figsize=(15, 5))
axes = fig.add_subplot(1, 3, 1)
scatter_plot_of_points(X_teacher, f'Original model (Euler ODE, {n_steps} NFE)', axes)
axes = fig.add_subplot(1, 3, 2)
scatter_plot_of_points(X_student, f'Distilled (Euler ODE, {n_steps} NFE)', axes)
```

```
 Original model (Euler ODE, 6 NFE)
Precision: 0.7655
Normalized Entropy: 0.9950
Harmonic Mean (F-score): 0.8653

 Distilled (Euler ODE, 6 NFE)
Precision: 0.8925
Normalized Entropy: 0.9948
Harmonic Mean (F-score): 0.9409
```



Amazing!

We already get an F1 score around 0.94 with just one distilling iteration. This is great compared to our baseline (with Euler ODE) which achieves an F1 score of around 0.86. Let us see what happens if we apply this recursively and start at teacher_N = 48.

In [21]:
```
def set_up_and_perform_recursive_student_training(network_params, training_params, scheduler, teacher_vf, teacher_noise, p_dat
    start = time.time()
    iterations = training_params['iterations']
    student = copy.deepcopy(teacher_noise) # MLP_fn(network_params) # Can also uncomment this line to use a different MLP arch

    for i in range(iterations):
        # New trainer class (but this one is pretty similar to the one used in the assignment)
        trainer = distill_mlp_fn(student, scheduler, teacher_vf, training_params)
```

```
        student = trainer.train_network(p_data)

        X = p_data.sample(1000)
        mean_x1 = torch.mean(X, dim=0)
        vf_fn = VectorFieldFn(student, 'noise_pred', scheduler=scheduler, data_mean=mean_x1)
        teacher_vf = copy.deepcopy(vf_fn)
        training_params['teacher_N'] = training_params['teacher_N']//2


    end = time.time()
    print(f"Total training time {end - start:.4f} seconds")

    # Will return score and vector field as well
    score_fn = ScoreFn(student, 'noise_pred', scheduler=scheduler)

    return student, score_fn, vf_fn
```

In [ ]:

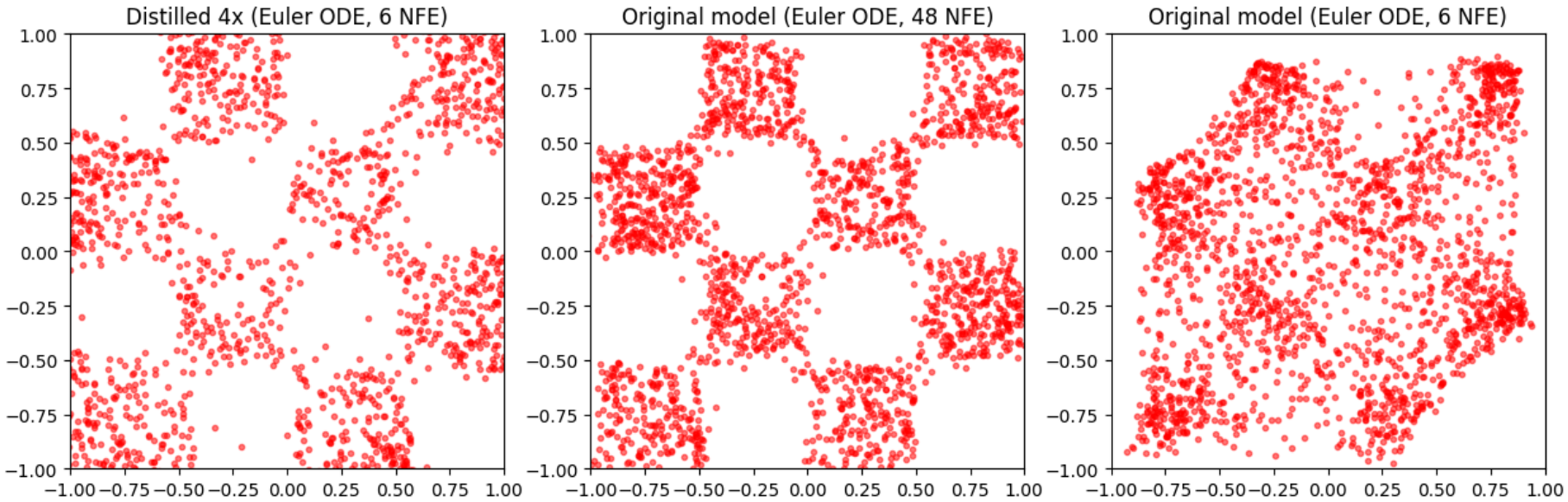
```
uniform_sampling = TimeSampling('uniform')
network_params = {'L': 4, 'h': 64}
training_params = {'n_batch': 1024, 'n_update': 2000, 'time_sampling': uniform_sampling, 'teacher_N': 48, 'iterations': 4}

temp_student_noise, temp_student_score, temp_student_vf = set_up_and_perform_recursive_student_training(
    network_params, training_params, linear_scheduler,
    teacher_vf=test_vf_fn, teacher_noise=test_noise_pred_fn, p_data=p_data
)
```

```
[Distill step 1] loss = 0.0000085
[Distill step 1000] loss = 0.0000049
[Distill step 1999] loss = 0.0000044
[Distill step 1] loss = 0.0000287
[Distill step 1000] loss = 0.0000147
[Distill step 1999] loss = 0.0000026
[Distill step 1] loss = 0.0003128
[Distill step 1000] loss = 0.0000111
[Distill step 1999] loss = 0.0000081
[Distill step 1] loss = 0.0034022
[Distill step 1000] loss = 0.0000613
[Distill step 1999] loss = 0.0000671
Total training time 85.3934 seconds
```

In [28]:

```python
# Cell that plots distilled and original sampling techniques next to each other
student_gen = GenerateSamples(update_step_euler_ode, temp_student_vf, sde_flag=False)
n_try = 2000
n_steps = 6
X_student = student_gen.generate(X_0, n_steps=n_steps)

print(f'\n Distilled 4x (Euler ODE, {n_steps} NFE)')
measure_performance(student_gen, p_data, n_try, n_steps = n_steps)

fig = plt.figure(figsize=(15, 5))
axes = fig.add_subplot(1, 3, 1)
scatter_plot_of_points(X_student, f'Distilled 4x (Euler ODE, {n_steps} NFE)', axes)

teacher_gen = GenerateSamples(update_step_euler_ode, test_vf_fn, sde_flag=False)
n_steps = 48
X_teacher = teacher_gen.generate(X_0, n_steps=n_steps)

print(f'\n Original model (Euler ODE, {n_steps} NFE)')
measure_performance(teacher_gen, p_data, n_try, n_steps = n_steps)

axes = fig.add_subplot(1, 3, 2)
scatter_plot_of_points(X_teacher, f'Original model (Euler ODE, {n_steps} NFE)', axes)

n_steps = 6
X_teacher_2 = teacher_gen.generate(X_0, n_steps=n_steps)
axes = fig.add_subplot(1, 3, 3)
scatter_plot_of_points(X_teacher_2, f'Original model (Euler ODE, {n_steps} NFE)', axes)
print(f'\n Original model (Euler ODE, {n_steps} NFE)')
temp = measure_performance(teacher_gen, p_data, n_try, n_steps = n_steps)
```

```
 Distilled 4x (Euler ODE, 6 NFE)
Precision: 0.8685
Normalized Entropy: 0.9735
Harmonic Mean (F-score): 0.9180

 Original model (Euler ODE, 48 NFE)
Precision: 0.9470
Normalized Entropy: 0.9953
Harmonic Mean (F-score): 0.9705

 Original model (Euler ODE, 6 NFE)
Precision: 0.7625
Normalized Entropy: 0.9985
Harmonic Mean (F-score): 0.8647
```



Doing 4 iterations we do not get as good performance (still >0.915 though) but the perceptual quality is now better (in my opinion it looks more like a checkerboard).

Let us also see how we do for 3 NFE, meaning we will do 5 iterations of distilling if we start from teacher_N = 48.

In [29]:
```
uniform_sampling = TimeSampling('uniform')
network_params = {'L': 4, 'h': 64}
```

```python
training_params = {'n_batch': 1024, 'n_update': 2000, 'time_sampling': uniform_sampling, 'teacher_N': 48, 'iterations': 5}

temp_student_noise, temp_student_score, temp_student_vf = set_up_and_perform_recursive_student_training(
    network_params, training_params, linear_scheduler,
    teacher_vf=test_vf_fn, teacher_noise=test_noise_pred_fn, p_data=p_data
)
```

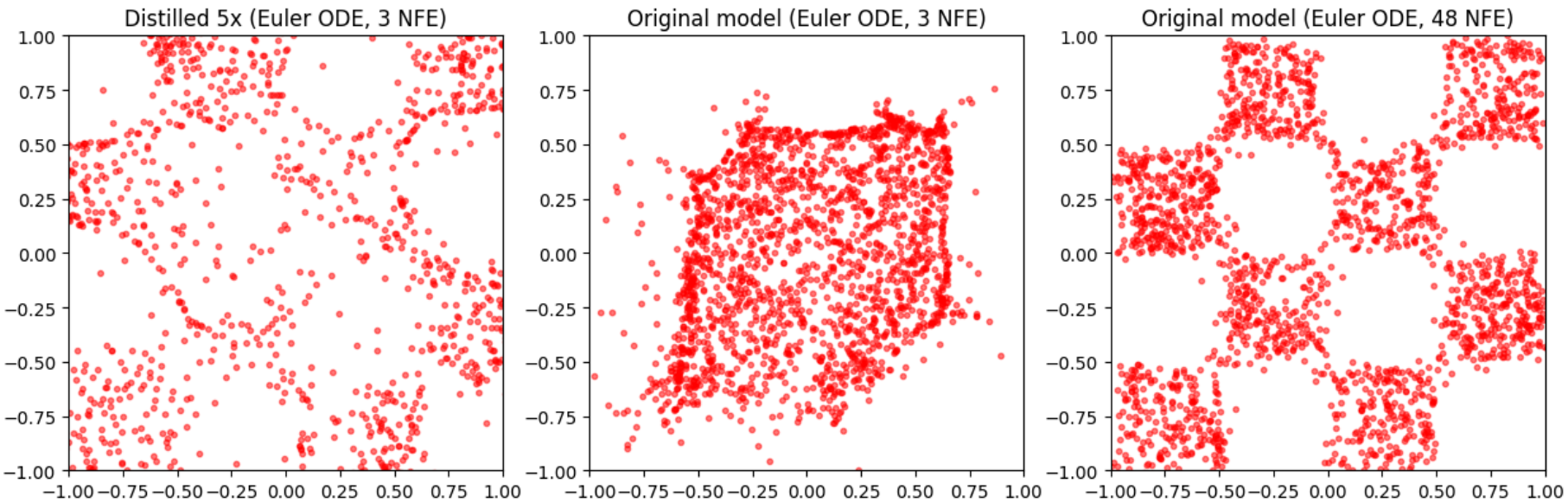
```
[Distill step 1] loss = 0.0000080
[Distill step 1000] loss = 0.0000073
[Distill step 1999] loss = 0.0000200
[Distill step 1] loss = 0.0000377
[Distill step 1000] loss = 0.0000014
[Distill step 1999] loss = 0.0000042
[Distill step 1] loss = 0.0003378
[Distill step 1000] loss = 0.0000152
[Distill step 1999] loss = 0.0000209
[Distill step 1] loss = 0.0034508
[Distill step 1000] loss = 0.0000586
[Distill step 1999] loss = 0.0000284
[Distill step 1] loss = 0.0345814
[Distill step 1000] loss = 0.0001505
[Distill step 1999] loss = 0.0001110
Total training time 114.2485 seconds
```

In [31]:

```python
# Cell that plots distilled and original sampling techniques next to each other
student_gen = GenerateSamples(update_step_euler_ode, temp_student_vf, sde_flag=False)
n_try = 2000
n_steps = 3
X_student = student_gen.generate(X_0, n_steps=n_steps)

print(f'\n Distilled 5x (Euler ODE, {n_steps} NFE)')
measure_performance(student_gen, p_data, n_try, n_steps = n_steps)

fig = plt.figure(figsize=(15, 5))
axes = fig.add_subplot(1, 3, 1)
scatter_plot_of_points(X_student, f'Distilled 5x (Euler ODE, {n_steps} NFE)', axes)

teacher_gen = GenerateSamples(update_step_euler_ode, test_vf_fn, sde_flag=False)
X_teacher_2 = teacher_gen.generate(X_0, n_steps=n_steps)
print(f'\n Original model (Euler ODE, {n_steps} NFE)')
measure_performance(teacher_gen, p_data, n_try, n_steps = n_steps)
```

```python
axes = fig.add_subplot(1, 3, 2)
scatter_plot_of_points(X_teacher_2, f'Original model (Euler ODE, {n_steps} NFE)', axes)

n_steps = 48
X_teacher = teacher_gen.generate(X_0, n_steps=n_steps)

print(f'\n Original model (Euler ODE, {n_steps} NFE)')
measure_performance(teacher_gen, p_data, n_try, n_steps = n_steps)

axes = fig.add_subplot(1, 3, 3)
scatter_plot_of_points(X_teacher, f'Original model (Euler ODE, {n_steps} NFE)', axes)
```

```
 Distilled 5x (Euler ODE, 3 NFE)
Precision: 0.7705
Normalized Entropy: 0.9243
Harmonic Mean (F-score): 0.8404

 Original model (Euler ODE, 3 NFE)
Precision: 0.4860
Normalized Entropy: 0.9219
Harmonic Mean (F-score): 0.6365

 Original model (Euler ODE, 48 NFE)
Precision: 0.9465
Normalized Entropy: 0.9984
Harmonic Mean (F-score): 0.9718
```



Well, for 3 NFE it does not work too well but still way better than no distillation (F1 ≈ 0.84 compared to F1 ≈ 0.64).

Reflection/toughts

It was fun and interesting to play around with this simple example of progressive distillation. To get better results one would preferably change to a DDIM solver (like used in the paper) or something similar but it was cool to see that it worked quite ok with a simple Euler ODE. I could also have tried the progressive distillation with Euler-Murayama SGD sampling or looked at what results we could get when trained with the custom scheduler. But that is a mission for another day!

End of my A/B submission

Acknowledgements: Please note the structure of the pdf sampling and FM training of the neural network code is copied and/or adapted from https://github.com/eje24/iap-diffusion-labs/blob/main/solutions/lab_two_complete.ipynb and https://github.com/facebookresearch/flow_matching